Exemple d'utilisation du package levyBridge

Sommaire

Préparation **1**

Le pont seul **2**

I - Une trajectoire **4**
Gamma . . . 4
Wiener . . . 4
Inverse gaussien . . . 5
Variance-gamma . . . 6
Meixner . . . 7

II - Plusieurs trajectoires **9**
Gamma . . . 9
Wiener . . . 9
Inverse gaussien . . . 10
Variance-gamma . . . 11
Meixner . . . 12
Mêmes instants pour toutes les trajectoires (matrice) . . . 14

III - Données réelles : lasers GaAs **16**

Préparation

Installation des packages depuis GitHub : `levyBridge` pour les ponts, `DegradSim` pour simuler des trajectoires et `levyfit` pour estimer les paramètres.

```
remotes::install_github("rytracetito/levy_packages", subdir = "levyBridge")
remotes::install_github("rytracetito/levy_packages", subdir = "DegradSim")
remotes::install_github("rytracetito/levy_packages", subdir = "levyfit")
```

Le package a deux familles de fonctions :

- `x_bridge(t_A, x_A, t_B, x_B, t, theta, n)` tire X_t sachant $X_{t_A} = x_A$ et $X_{t_B} = x_B$, selon la loi du pont du processus ;
- `x_resampling(x, theta, method)` rééchantillonne des trajectoires observées à des instants irréguliers $0 = t_0 < \cdots < t_N$ sur une grille régulière, ici $t'_i = i\, t_N/N$ (méthode `"Bridge"` de Grall-Maës, 2012), en appelant `x_bridge` en chaque point de la grille.

Dans les deux cas, `theta` est le vecteur nommé de tous les paramètres du processus.

Paramètres utilisés :

Processus	Pont	Rééchantillonnage	`theta`	Utilisés par le pont
Gamma	`gamma_bridge`	`gamma_resampling`	`a = 2, b = 3`	`a`

Processus	Pont	Rééchantillonnage	`theta`	Utilisés par le pont
Wiener	`wiener_bridge`	`wiener_resampling`	`mu = 0.5, sigma = 1.5`	`sigma`
Inverse gaussien	`ig_bridge`	`ig_resampling`	`mu = 1, eta = 3`	`eta`
Variance-gamma	`vg_bridge`	`vg_resampling`	`theta = 0.5, sigma = 1.5, nu = 2`	`theta, sigma, nu`
Meixner	`meixner_bridge`	`meixner_resampling`	`a = 1, b = 0.3, d = 1.5, m = 0.2`	`a, d, m`

Le pont seul

Les fonctions `x_bridge(t_A, x_A, t_B, x_B, t, theta, n)` tirent n valeurs de X_t sachant $X_{t_A} = x_A$ et $X_{t_B} = x_B$. Cinq tirages de X_2 sachant $X_1 = 3$ et $X_4 = 6$, pour chaque processus :

```
set.seed(1)
rbind(gamma   = gamma_bridge(1, 3, 4, 6, 2, theta = c(a = 2, b = 3), n = 5),
      wiener  = wiener_bridge(1, 3, 4, 6, 2, theta = c(mu = 0.5, sigma = 1.5), n = 5),
      ig      = ig_bridge(1, 3, 4, 6, 2, theta = c(mu = 1, eta = 3), n = 5),
      vg      = vg_bridge(1, 3, 4, 6, 2, theta = c(theta = 0.5, sigma = 1.5, nu = 2), n = 5),
      meixner = meixner_bridge(1, 3, 4, 6, 2, theta = c(a = 1, b = 0.3, d = 1.5, m = 0.2), n = 5))
```

```
##             [,1]     [,2]     [,3]     [,4]     [,5]
## gamma   4.107916 4.049250 3.795927 3.905308 4.002051
## wiener  2.484331 3.725329 4.462213 4.163303 4.984927
## ig      4.027097 3.973720 3.706780 4.235324 3.892253
## vg      2.964580 3.807249 3.611558 2.683493 4.680622
## meixner 4.669200 3.638721 4.909379 4.161492 5.459250
```

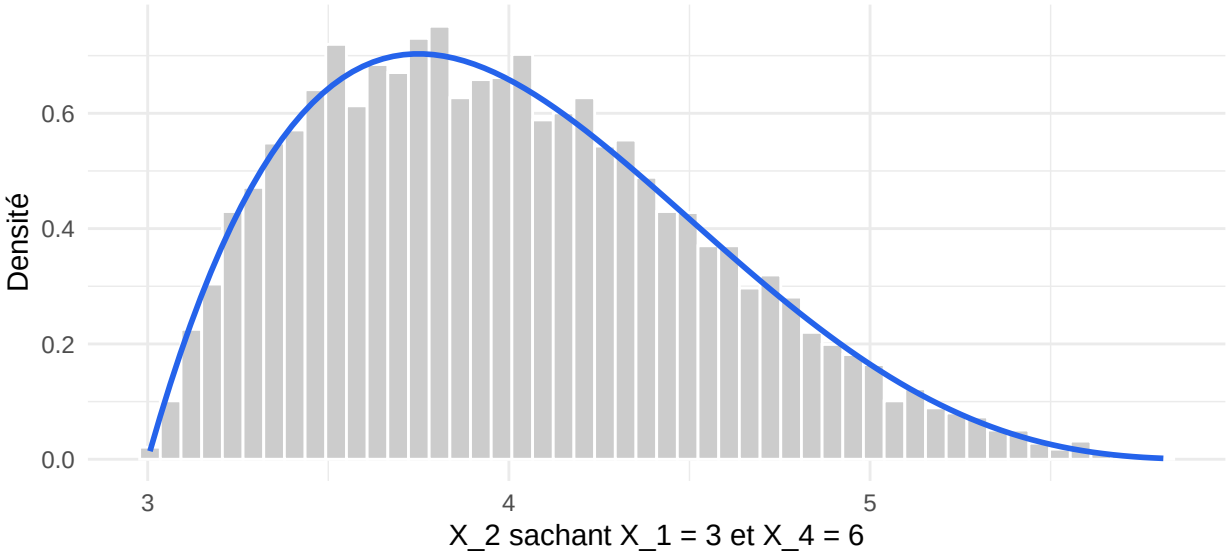
Pour les processus croissants (gamma et inverse gaussien), les valeurs restent entre 3 et 6 ; pour les autres, elles peuvent en sortir.

Pour le processus gamma, $X_2 = 3 + 3\,Y$ avec $Y \sim \text{Beta}(2, 4)$: l'histogramme de 10 000 tirages suit cette densité.

```
set.seed(1)
gamma_bridge(t_A = 1, x_A = 3, t_B = 4, x_B = 6, t = 2, theta = c(a = 2, b = 3), n = 5)
```

```
## [1] 4.107916 4.049250 3.795927 3.905308 4.002051
```

```
y <- gamma_bridge(1, 3, 4, 6, 2, theta = c(a = 2, b = 3), n = 10000)
ggplot(data.frame(y), aes(x = y)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50, fill = "grey80", colour = "white") +
  stat_function(fun = function(v) dbeta((v - 3) / 3, 2, 4) / 3, colour = "#2563EB", linewidth = 1) +
  labs(x = "X_2 sachant X_1 = 3 et X_4 = 6", y = "Densité") +
  theme_minimal()
```


Densité
0.6
0.4
0.2
0.0
3
4
5
X_2 sachant X_1 = 3 et X_4 = 6

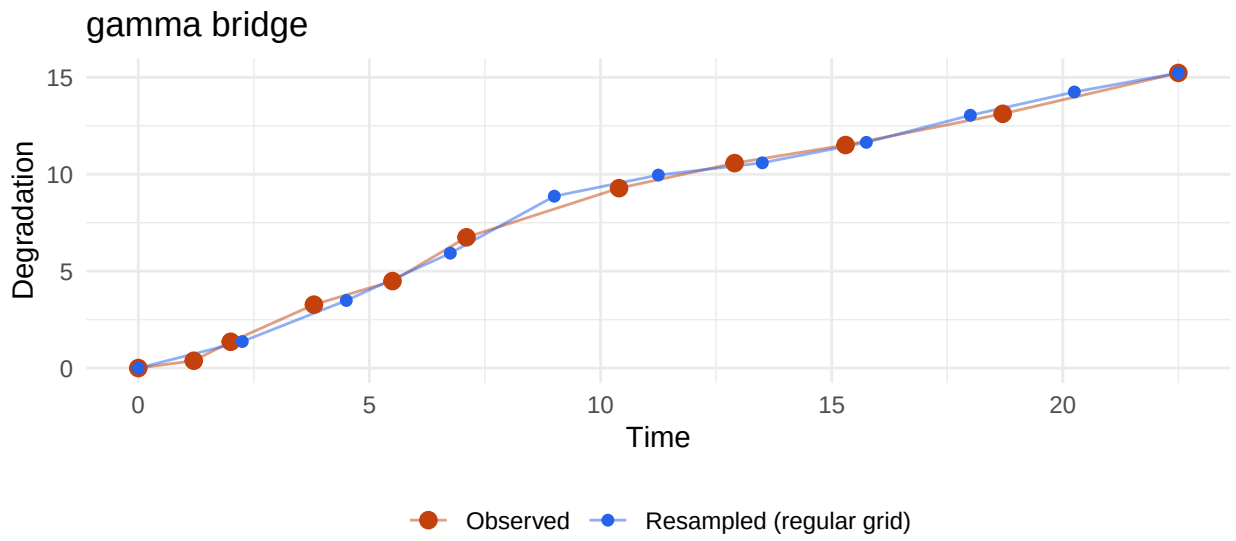
I - Une trajectoire

Une trajectoire observée à 11 instants irréguliers entre 0 et 22.5 ; la grille régulière est 0, 2.25, 4.5, . . . , 22.5.

```
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
```

Gamma

```
set.seed(1)
x <- sim_gamma(t, a = 2, b = 3)
r <- gamma_resampling(x, theta = c(a = 2, b = 3))
plot_bridge(r)
```

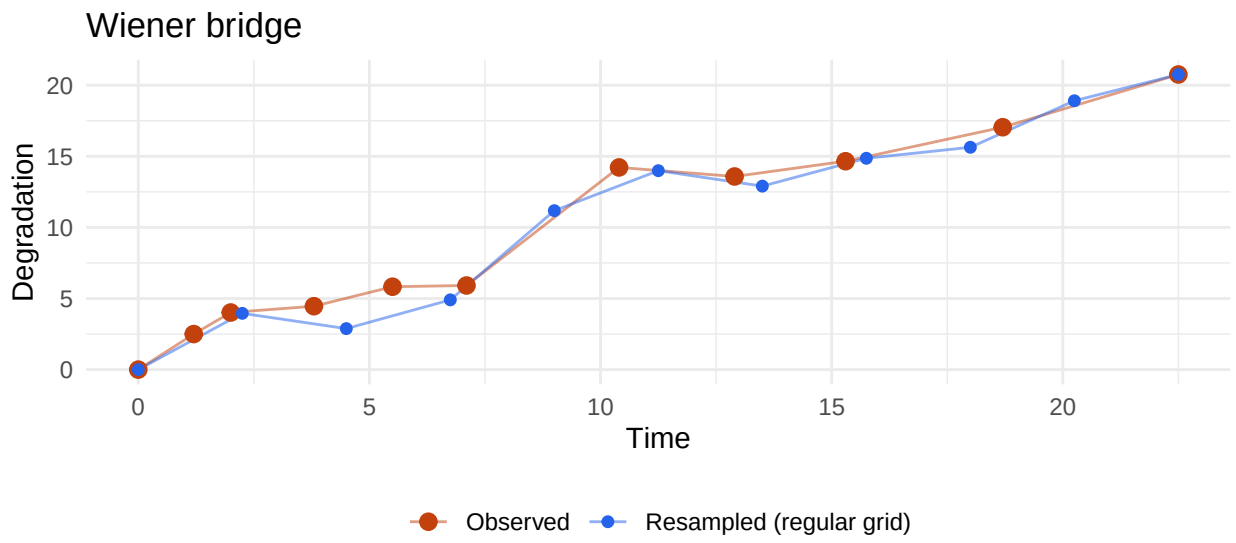



```
r[[1]]
```

```
##        t         X
## 1   0.00  0.000000
## 2   2.25  1.369233
## 3   4.50  3.487545
## 4   6.75  5.928483
## 5   9.00  8.867842
## 6  11.25  9.958656
## 7  13.50 10.591760
## 8  15.75 11.647523
## 9  18.00 13.043327
## 10 20.25 14.246231
## 11 22.50 15.229411
```

Wiener

```
x <- sim_wiener(t, mu = 0.5, sigma = 1.5)
r <- wiener_resampling(x, theta = c(mu = 0.5, sigma = 1.5))
plot_bridge(r)
```

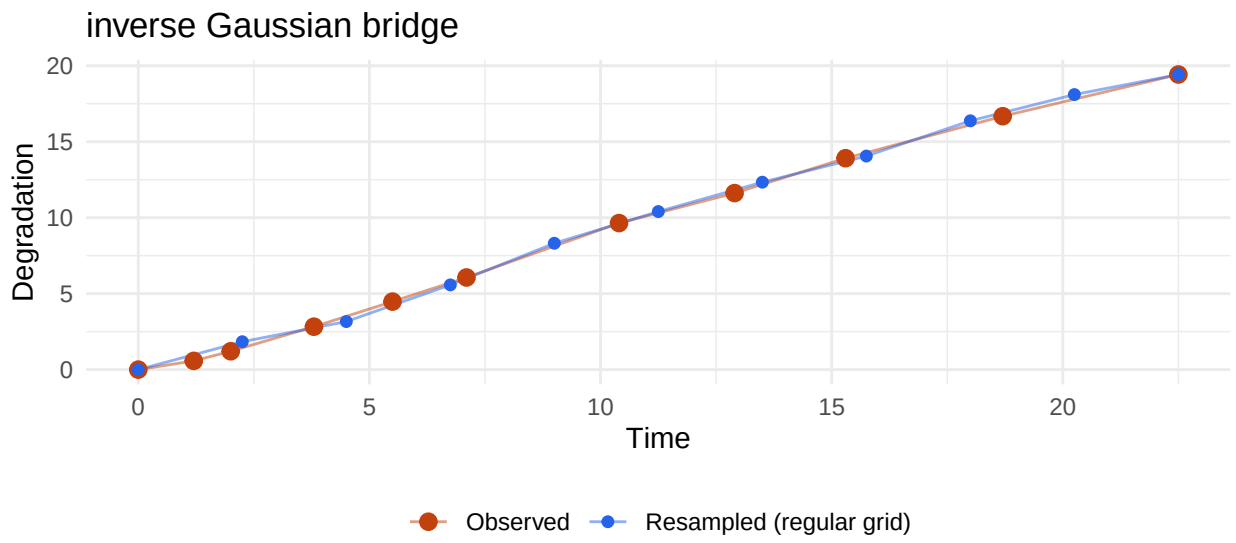



```
r[[1]]
```

```
##         t         X
## 1    0.00  0.000000
## 2    2.25  3.958011
## 3    4.50  2.882948
## 4    6.75  4.905171
## 5    9.00 11.178257
## 6   11.25 13.988643
## 7   13.50 12.902696
## 8   15.75 14.861571
## 9   18.00 15.633992
## 10 20.25 18.905931
## 11 22.50 20.753187
```

Inverse gaussien

```
x <- sim_ig(t, mu = 1, eta = 3)
r <- ig_resampling(x, theta = c(mu = 1, eta = 3))
plot_bridge(r)
```

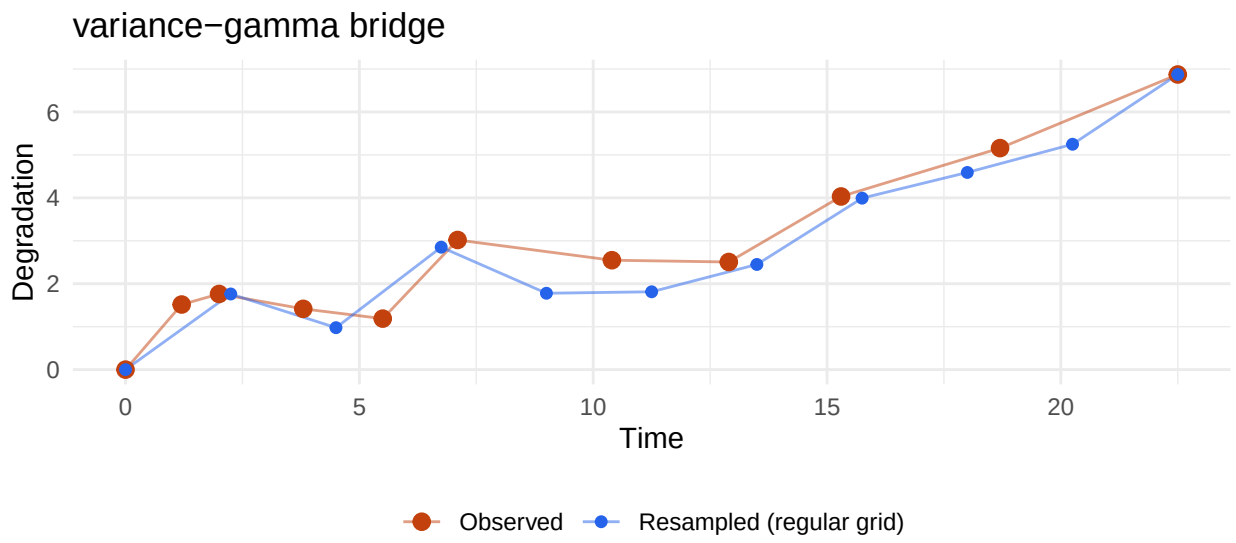



```
r[[1]]
```

```
##         t         X
## 1    0.00  0.000000
## 2    2.25  1.835712
## 3    4.50  3.162932
## 4    6.75  5.574668
## 5    9.00  8.315545
## 6   11.25 10.401009
## 7   13.50 12.335177
## 8   15.75 14.055437
## 9   18.00 16.374405
## 10 20.25 18.100838
## 11 22.50 19.422710
```

Variance-gamma

```
x <- sim_vg(t, theta = 0.5, sigma = 1.5, nu = 2)
r <- vg_resampling(x, theta = c(theta = 0.5, sigma = 1.5, nu = 2))
plot_bridge(r)
```

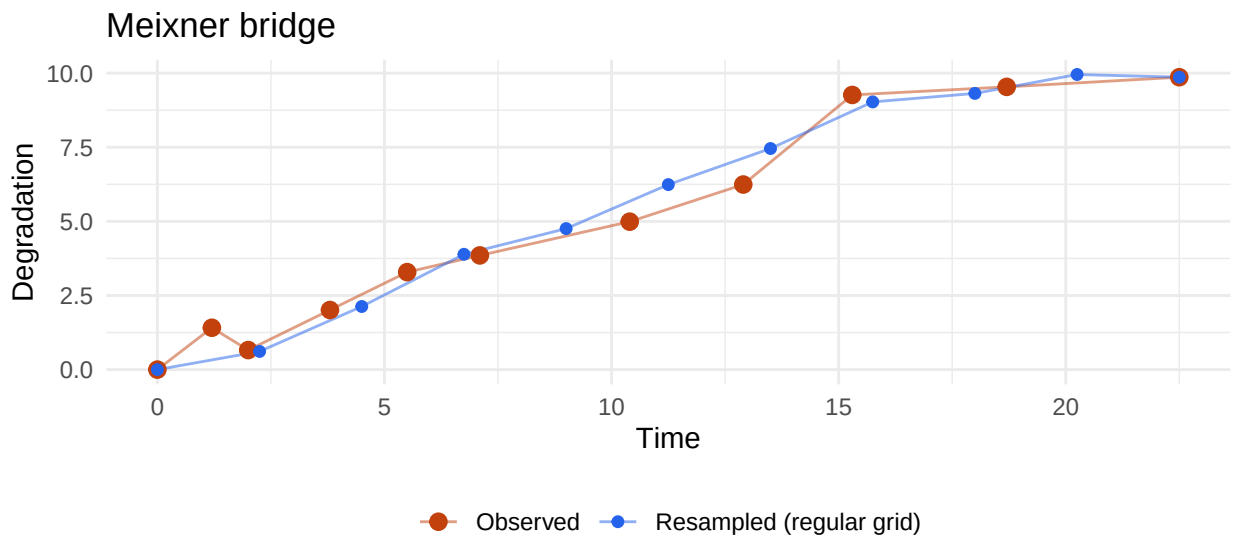



```
r[[1]]
```

```
##        t         X
## 1   0.00 0.0000000
## 2   2.25 1.7612991
## 3   4.50 0.9766158
## 4   6.75 2.8511577
## 5   9.00 1.7784044
## 6  11.25 1.8125508
## 7  13.50 2.4487399
## 8  15.75 3.9927572
## 9  18.00 4.5914357
## 10 20.25 5.2479706
## 11 22.50 6.8727294
```

Meixner

```
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2)
r <- meixner_resampling(x, theta = c(a = 1, b = 0.3, d = 1.5, m = 0.2))
plot_bridge(r)
```




```
r[[1]]
```

```
##         t         X
## 1    0.00 0.0000000
## 2    2.25 0.6133239
## 3    4.50 2.1286482
## 4    6.75 3.8893977
## 5    9.00 4.7560786
## 6   11.25 6.2395696
## 7   13.50 7.4572831
## 8   15.75 9.0291142
## 9   18.00 9.3159913
## 10 20.25 9.9535774
## 11 22.50 9.8620213
```

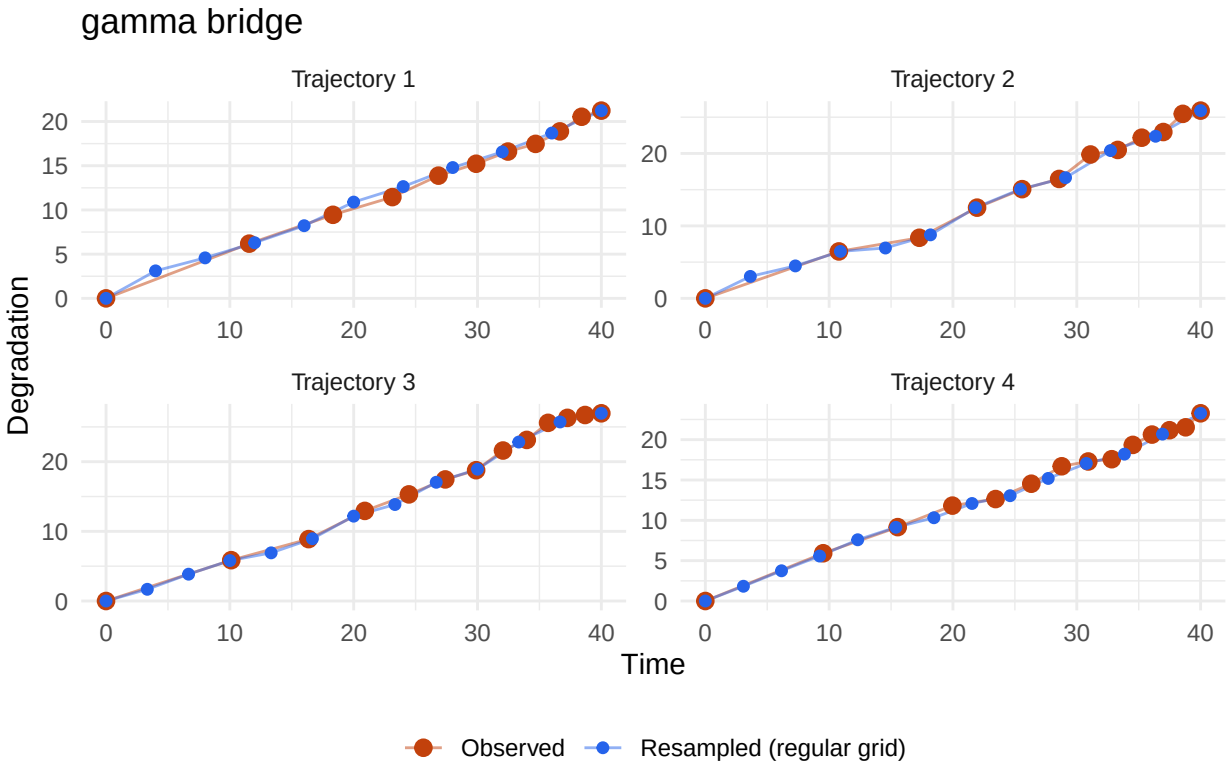
II - Plusieurs trajectoires

20 trajectoires inspectées entre 0 et 40, de plus en plus souvent. Chaque trajectoire a ses propres instants d'inspection : 11 instants pour la première, 12 pour la deuxième, et ainsi de suite jusqu'à 30 pour la dernière. Le pont reconstruit chaque trajectoire sur sa propre grille régulière.

```
plan <- inspection_times("decreasing", n = 10:29, n_traj = 20, tmax = 40)
```

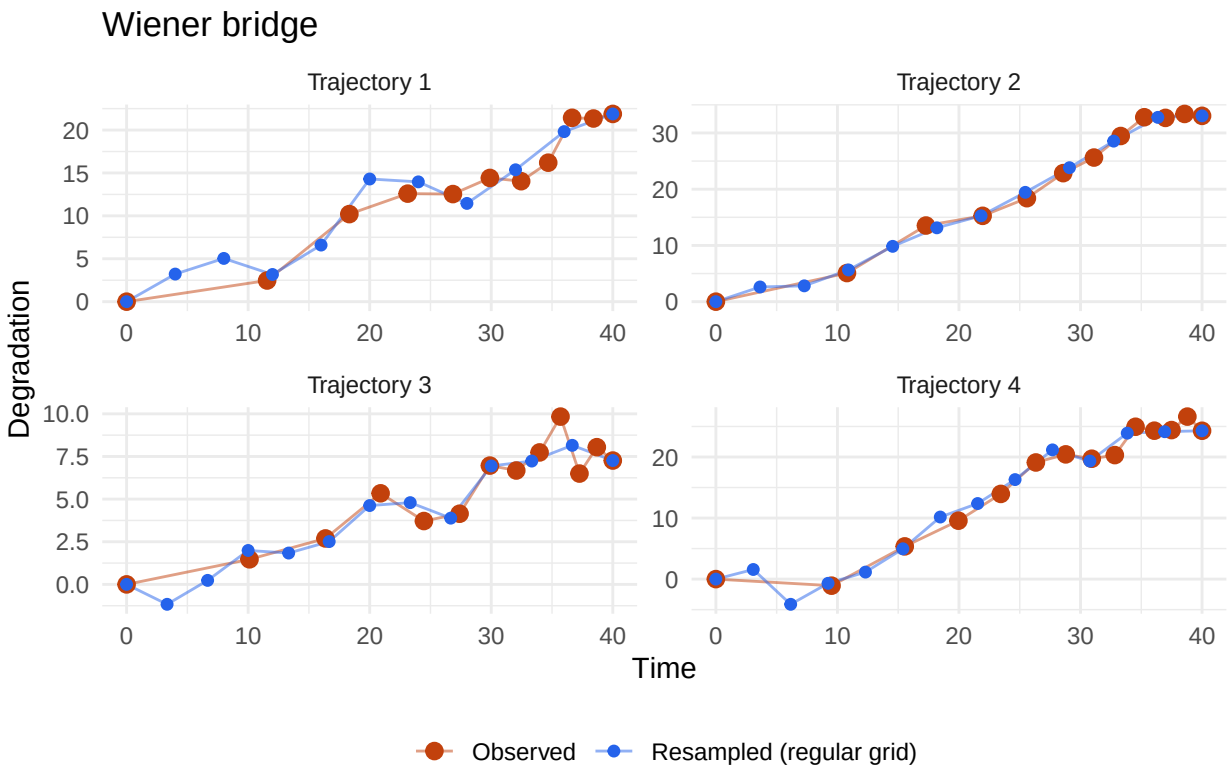
Gamma

```
set.seed(2)
r <- gamma_resampling(sim_gamma(plan, a = 2, b = 3), theta = c(a = 2, b = 3))
plot_bridge(r, trajectory = 1:4)
```



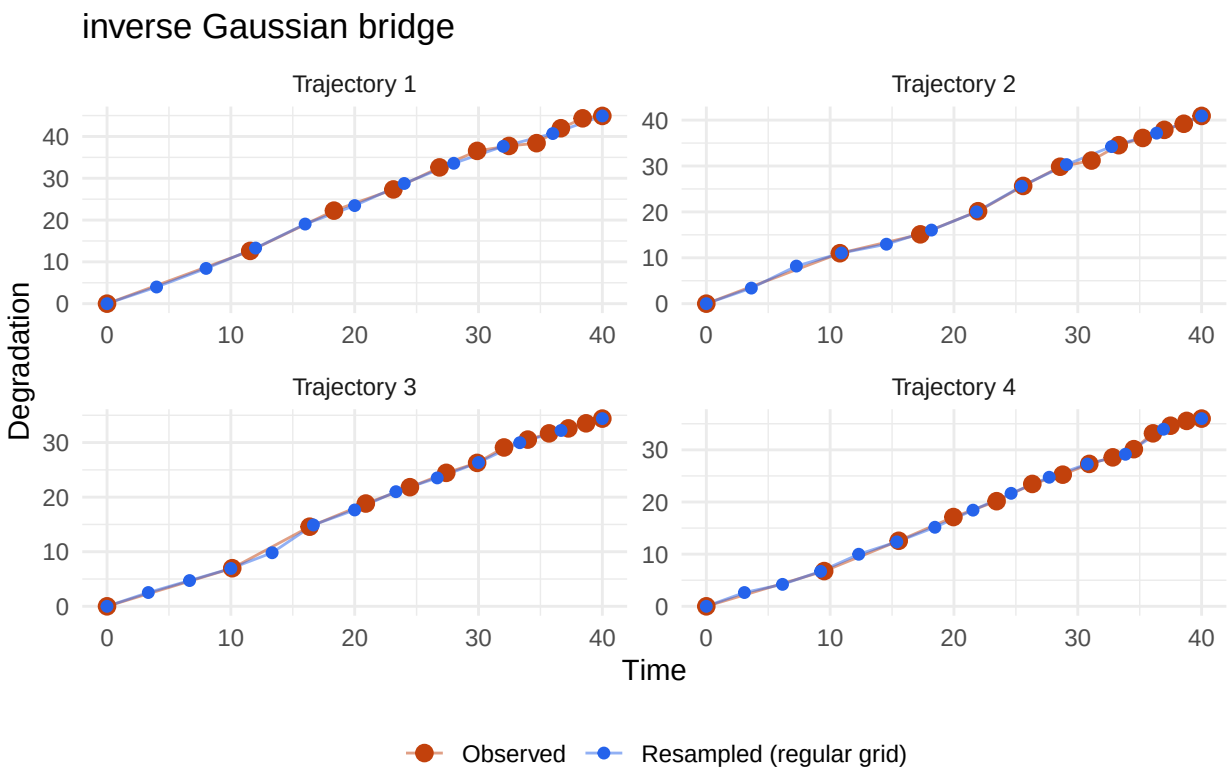
Wiener

```
r <- wiener_resampling(sim_wiener(plan, mu = 0.5, sigma = 1.5), theta = c(mu = 0.5, sigma = 1.5))
plot_bridge(r, trajectory = 1:4)
```



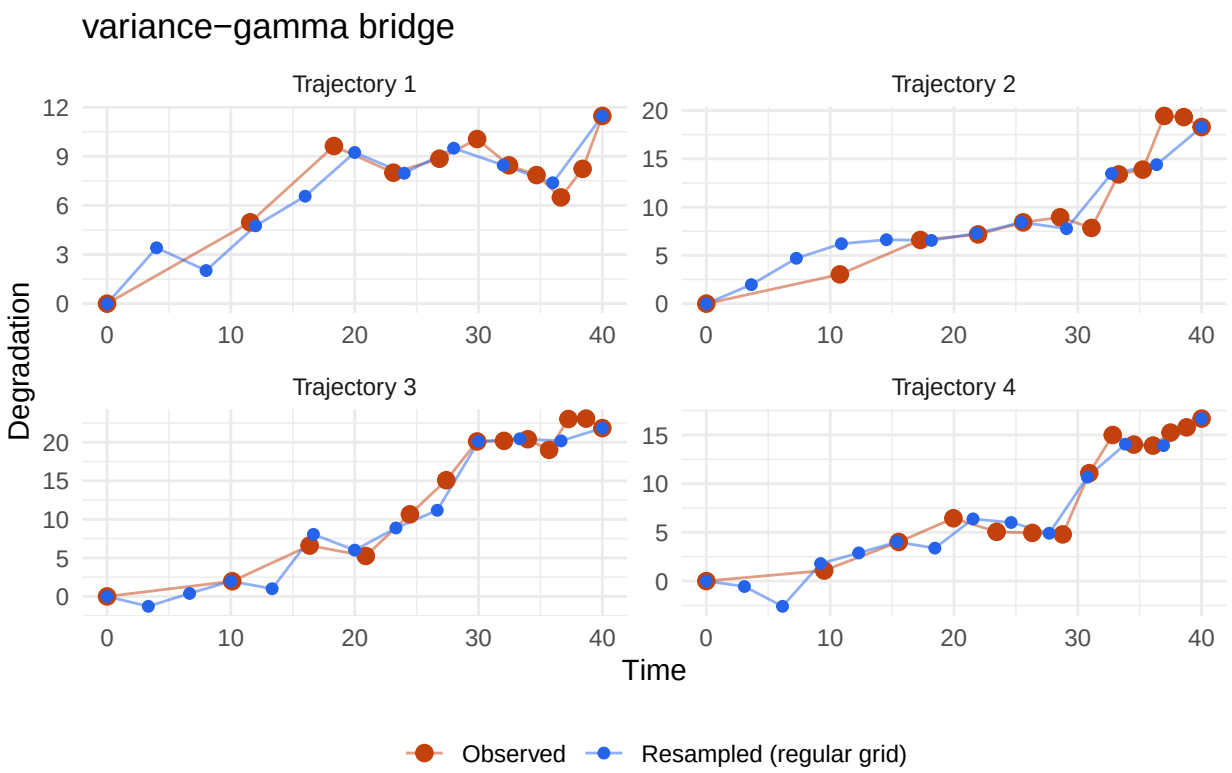
Inverse gaussien

```
r <- ig_resampling(sim_ig(plan, mu = 1, eta = 3), theta = c(mu = 1, eta = 3))
plot_bridge(r, trajectory = 1:4)
```



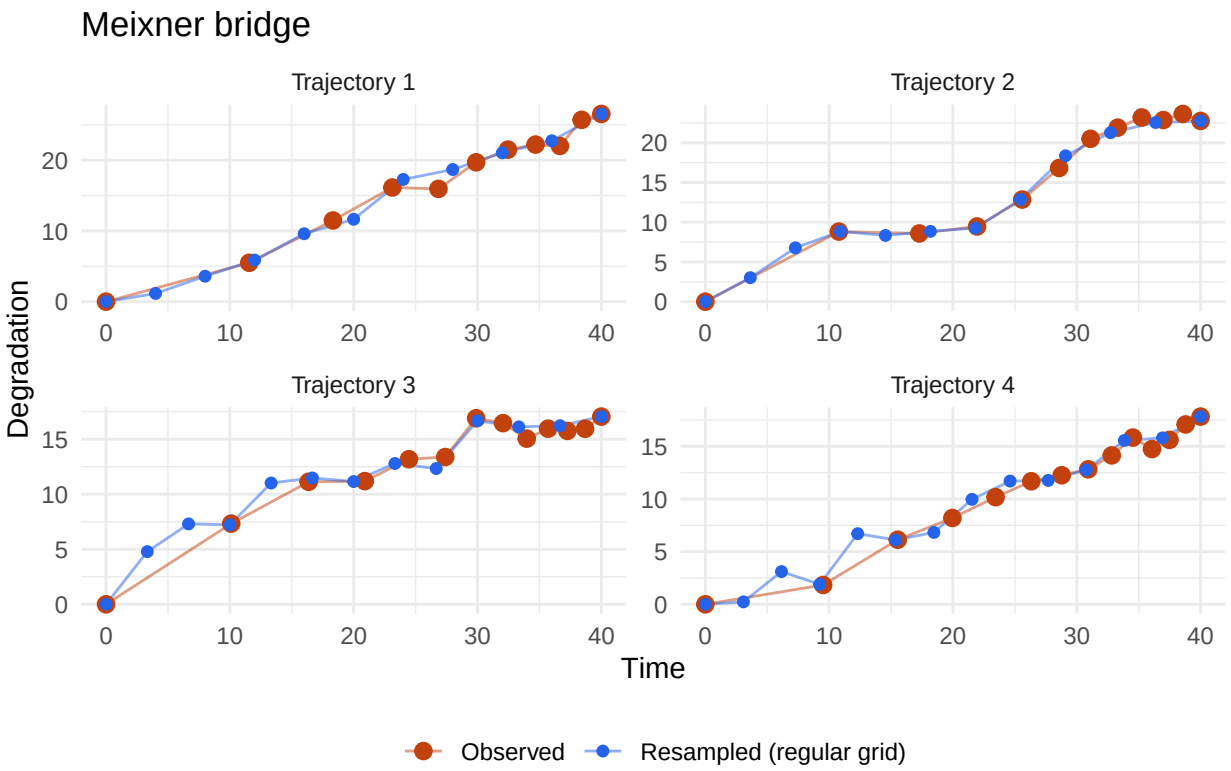
Variance-gamma

```
x <- sim_vg(plan, theta = 0.5, sigma = 1.5, nu = 2)
r <- vg_resampling(x, theta = c(theta = 0.5, sigma = 1.5, nu = 2))
plot_bridge(r, trajectory = 1:4)
```



Meixner

```
x <- sim_meixner(plan, a = 1, b = 0.3, d = 1.5, m = 0.2)
r <- meixner_resampling(x, theta = c(a = 1, b = 0.3, d = 1.5, m = 0.2))
plot_bridge(r, trajectory = 1:4)
```



Meixner avec les paramètres estimés

En pratique, les vraies valeurs des paramètres sont inconnues : on les estime d'abord sur les trajectoires observées avec `levyfit`, puis on construit le pont avec ces estimations. Le paramètre `b` est estimé mais n'est pas utilisé par le pont.

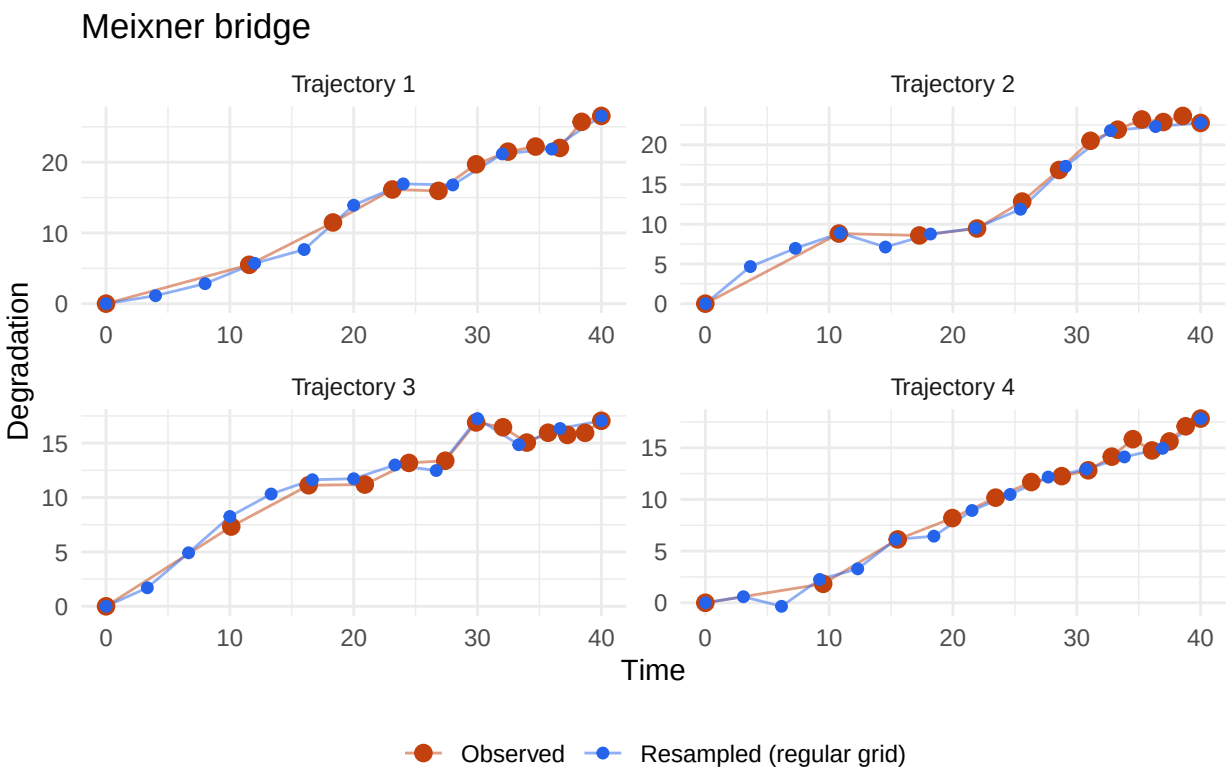
```
est <- meixner_mle(x)
est
```

```
##           a           b           d           m
##  0.65017924  0.47156738  3.20283665 -0.04074474
```

```
r_est <- meixner_resampling(x, theta = est)
r_est
```

```
## 20 trajectory(ies) resampled on a regular grid by the Meixner bridge (method "Bridge")
## parameters: a = 0.6502, b = 0.4716, d = 3.203, m = -0.04074
## time step teq: from 1.379 to 4 (one per trajectory, see attr(x, "teq"))
##
##   id  t         X
## 1  1  0  0.000000
## 2  1  4  1.129168
## 3  1  8  2.828458
## 4  1 12  5.699035
## 5  1 16  7.651401
## 6  1 20 13.916412
```

```
plot_bridge(r_est, trajectory = 1:4)
```



Mêmes instants pour toutes les trajectoires (matrice)

Quand toutes les trajectoires ont les mêmes instants d'inspection, `DegradSim` renvoie une matrice (colonne `t`, puis une colonne par trajectoire), acceptée telle quelle :

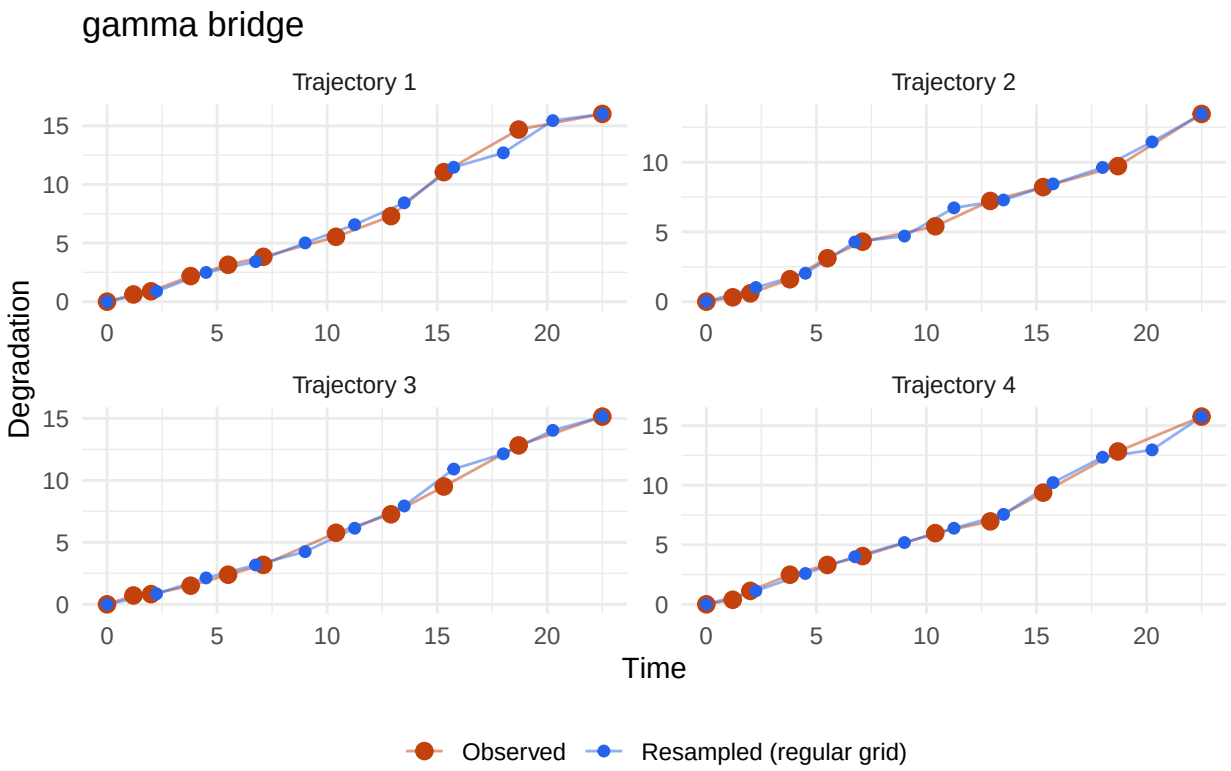
```
xm <- sim_gamma(t, a = 2, b = 3, n_traj = 5)
head(xm)
```

```
##         t        X1        X2        X3        X4        X5
## [1,] 0.0 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [2,] 1.2 0.6154554 0.3114827 0.7135821 0.3810535 0.5790518
## [3,] 2.0 0.8850816 0.5975521 0.8206202 1.1286029 0.9511820
## [4,] 3.8 2.1816978 1.6053126 1.5033262 2.4836508 3.7660051
## [5,] 5.5 3.1469204 3.1144941 2.3887160 3.3059025 4.9876854
## [6,] 7.1 3.8248944 4.3013766 3.1795768 4.0470699 6.4030033
```

```
r_m <- gamma_resampling(xm, theta = c(a = 2, b = 3))
attr(r_m, "teq")
```

```
## [1] 2.25 2.25 2.25 2.25 2.25
```

```
plot_bridge(r_m, trajectory = 1:4)
```


gamma bridge
Trajectory 1
Trajectory 2
Trajectory 3
Trajectory 4
Degradation
Time
Observed
Resampled (regular grid)

III - Données réelles : lasers GaAs

Les données `gaaslaser` donnent l'augmentation relative (en %) du courant de fonctionnement de 15 lasers, mesurée toutes les 250 heures jusqu'à 4000 heures. On ne garde que des inspections irrégulières, de moins en moins fréquentes, puis on estime le paramètre de forme du processus gamma avec `levyfit` et on reconstruit les trajectoires sur la grille régulière.

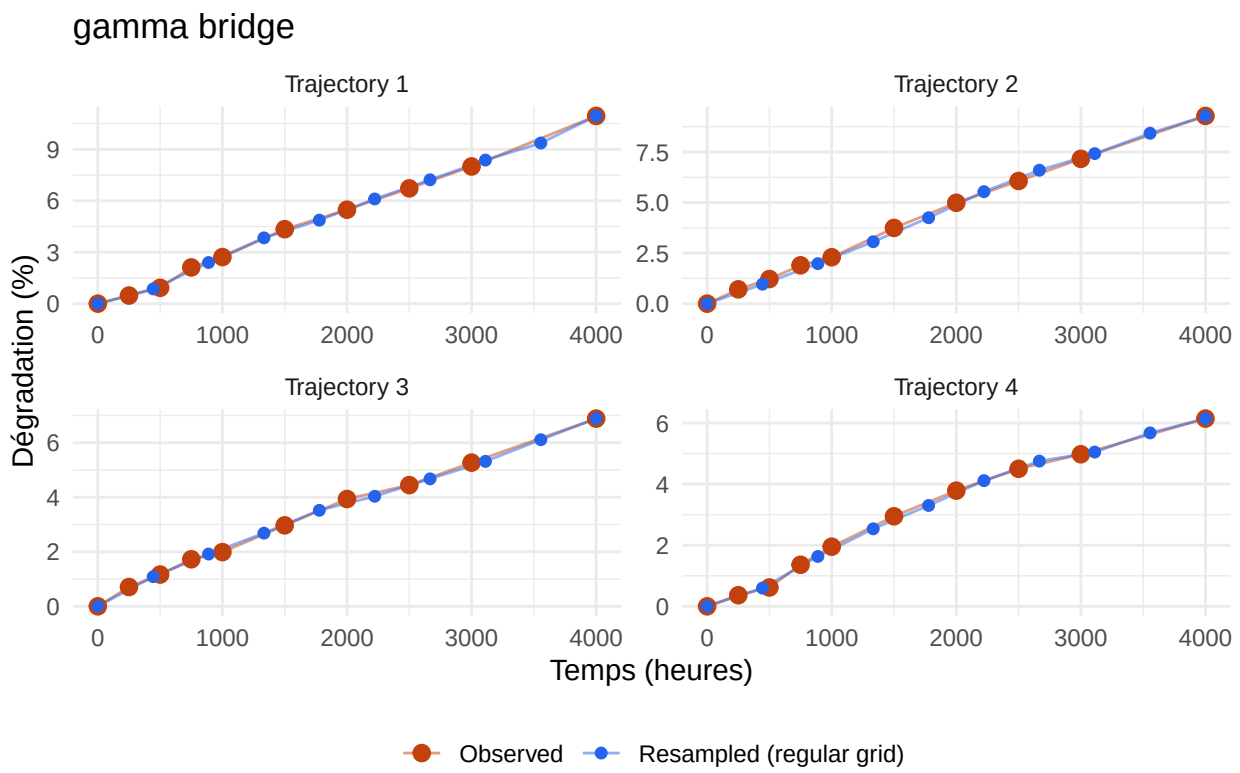
```
d <- SMRD.data::gaaslaser
temps <- c(0, 250, 500, 750, 1000, 1500, 2000, 2500, 3000, 4000)
lasers <- lapply(sort(unique(d$unit)), function(k) {
  s <- d[d$unit == k & d$hours %in% temps, ]
  data.frame(t = s$hours, X = s$increase)
})
est <- gamma_mle(lasers)
est
```

```
##           a           b
##  0.02166498 10.63099576
```

```
set.seed(3)
r <- gamma_resampling(lasers, theta = est)
r
```

```
## 15 trajectory(ies) resampled on a regular grid by the gamma bridge (method "Bridge")
## parameters: a = 0.02166, b = 10.63
## time step teq: 444.4
##
##   id         t         X
## 1  1    0.0000 0.0000000
## 2  1  444.4444 0.8610736
## 3  1  888.8889 2.3991991
## 4  1 1333.3333 3.8369925
## 5  1 1777.7778 4.8678986
## 6  1 2222.2222 6.1049463
```

```
plot_bridge(r, trajectory = 1:4) + labs(x = "Temps (heures)", y = "Dégradation (%)")
```

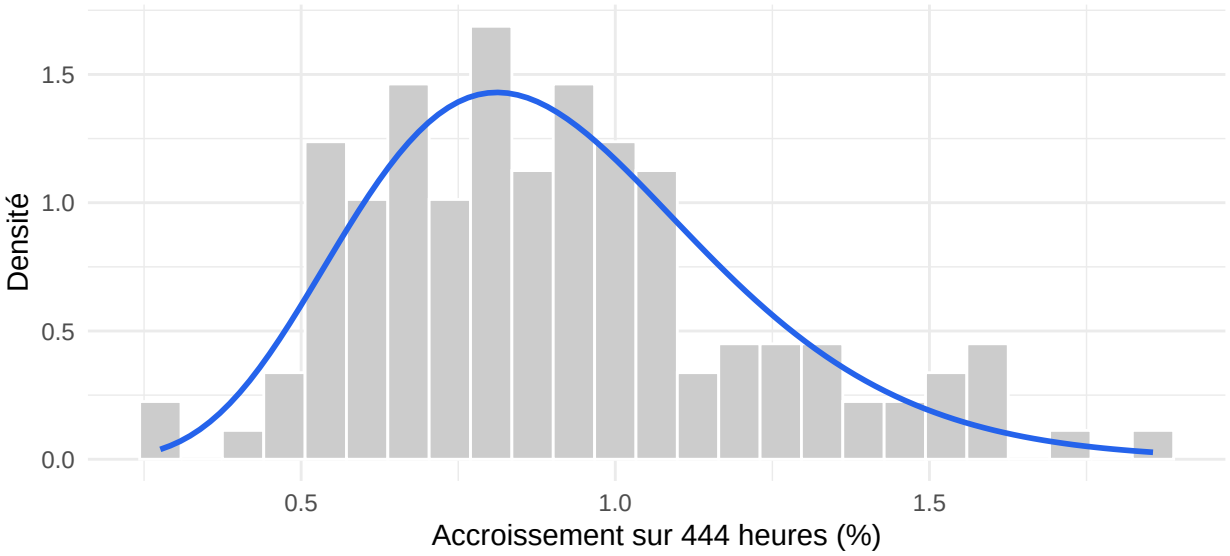


Accroissements reconstruits, sur une grille de pas $4000/9 \approx 444$ heures :

```
acc <- data.frame(dx = unlist(attr(r, "dx")))
head(acc)
```

```
##          dx
## 1 0.8610736
## 2 1.5381255
## 3 1.4377934
## 4 1.0309060
## 5 1.2370478
## 6 1.1176121
```

```
ggplot(acc, aes(x = dx)) +
  geom_histogram(aes(y = after_stat(density)), bins = 25, fill = "grey80", colour = "white") +
  stat_function(fun = dgamma, args = list(shape = est[["a"]] * attr(r, "teq")[1], rate = est[["b"]]),
                colour = "#2563EB", linewidth = 1) +
  labs(x = "Accroissement sur 444 heures (%)", y = "Densité") +
  theme_minimal()
```


Densité
1.5
1.0
0.5
0.0
0.5
1.0
1.5
Accroissement sur 444 heures (%)